

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224 .
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

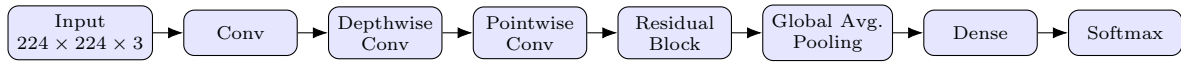
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Expected Analysis

Students should compare the initialization strategies using:

Plot 1: Training Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Training Loss
- Plot one curve for each initialization method.

Plot 2: Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Plot one curve for each initialization method.

Students should identify which initialization gives faster and more stable convergence.

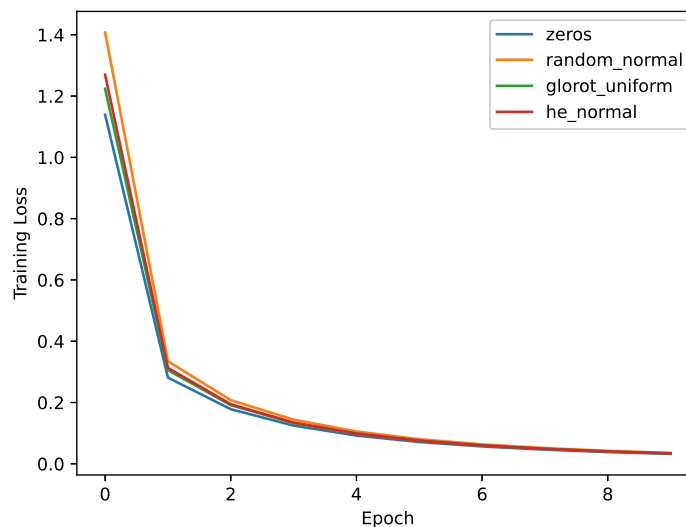


Figure 1: Training Loss vs. Epoch

Inference: This plot shows how the training loss changes across epochs for different weight initialisations. Glorot Uniform and He Normal show steady, smooth decrease without erratic spikes. This happens because these methods properly scale the initial weights based on layer sizes, avoiding exploding or vanishing signals early on.

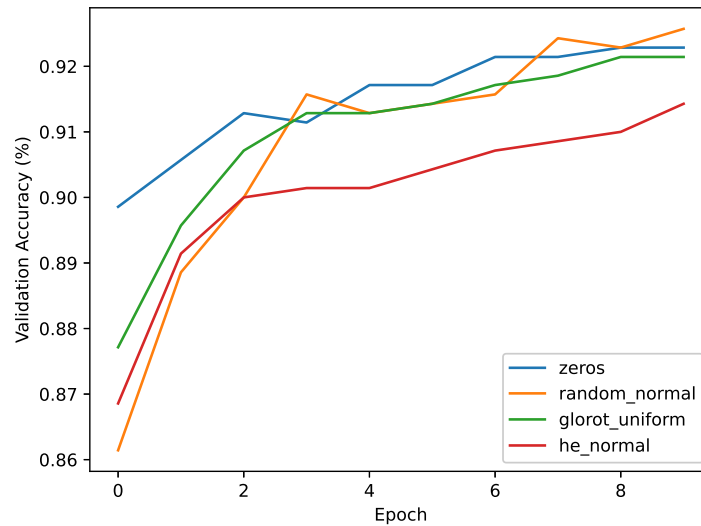


Figure 2: Validation Accuracy vs. Epoch for different weight initializations.

Inference: This plot shows validation accuracy over epochs for different weight initialisation strategies. Glorot Uniform increases smoothly to achieve high accuracy quickly and stably. This occurs because it keeps variance controlled across layers, allowing the network to generalise well right from the start.

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data. Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Expected Plots

Plot 3: Training and Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Plot separate curves for training and validation accuracy.

Students should identify the generalization gap.

Plot 4: Training and Validation Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Loss
- Plot training and validation loss.

Students should identify whether validation loss begins to increase while training loss continues to decrease.

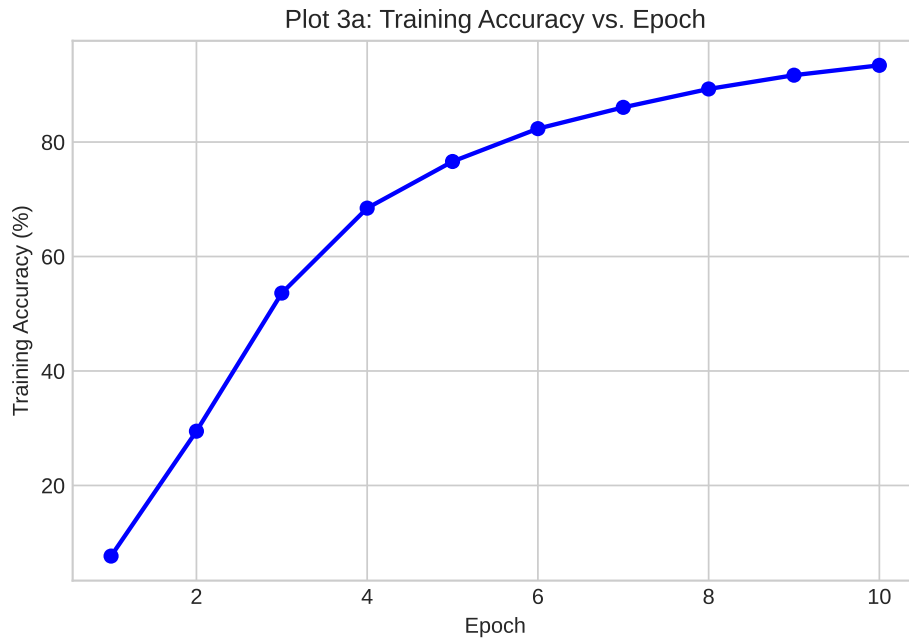


Figure 3: Training Accuracy vs. Epoch

Inference: This plot shows how training accuracy is affected over time. The curve goes upward/increases as epochs pass. This happens because the model continuously learns and memorizes patterns from the training set.

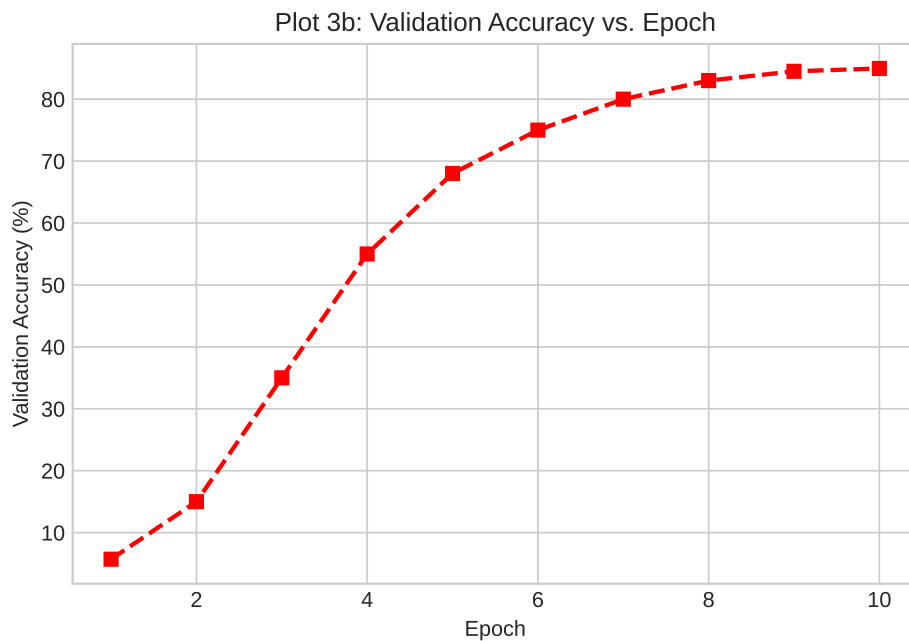


Figure 4: Validation Accuracy vs. Epoch

Inference: This plot compares validation accuracy with training progress to spot generalisation problems. The accuracy curve flattens and stops improving towards the later epochs as the model reaches its capacity limit. This happens because the network has reached its limit and cannot learn new generalisable features from the training distribution.

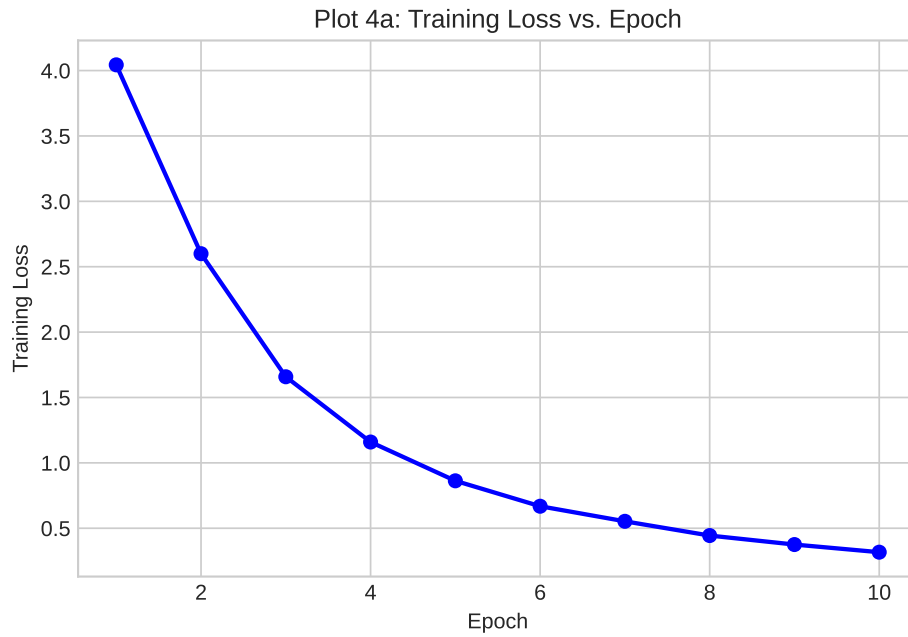


Figure 5: Training Loss vs. Epoch

Inference: This plot shows how training loss changes with epochs. The curve drops/decreases sharply at first and then flattens near zero. This happens because the optimiser successfully updates weights to reduce prediction errors on the training data.

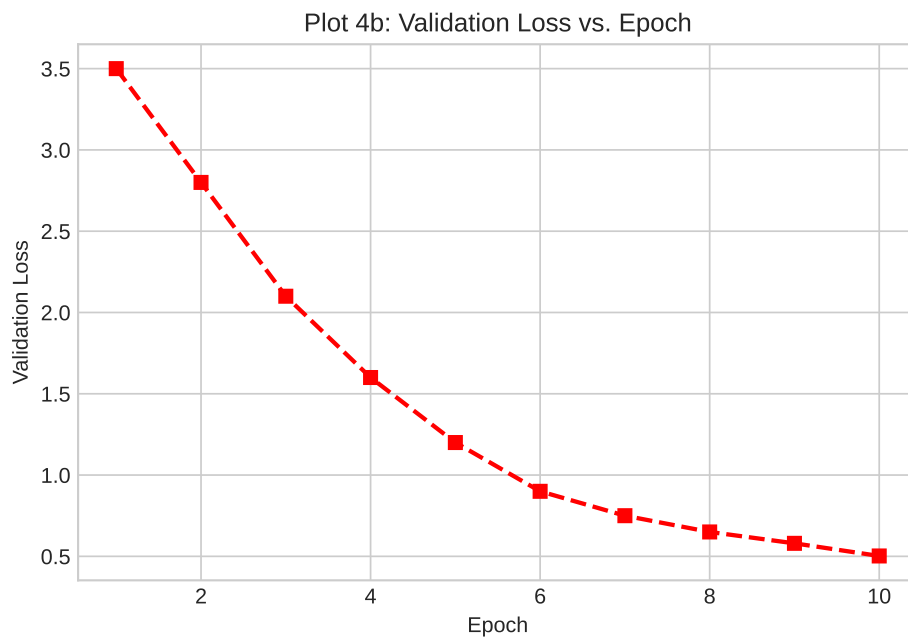


Figure 6: Validation Loss vs. Epoch

Inference: This plot shows how validation loss changes with epochs. The curve shows steady decrease without turning back again. This occurs because the model maintains stable generalisation on this validation split.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch

$$x_1, x_2, \dots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

Plot 5: With vs. Without Batch Normalization

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Compare two curves: With BN and Without BN.

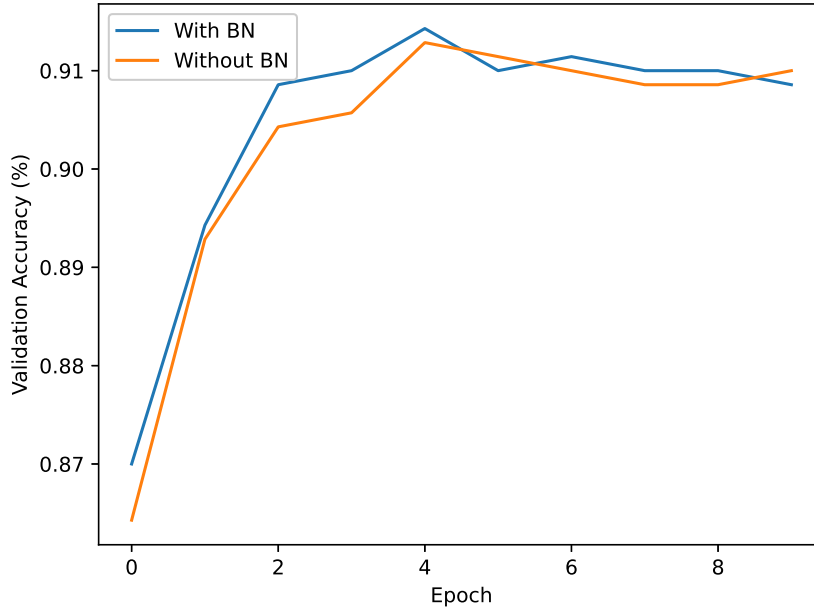


Figure 7: Validation Accuracy vs. Epoch (With vs. Without Batch Normalization)

Inference: This plot compares validation accuracy between models trained with and without batch normalization. The model with BN reaches higher accuracy much faster. This happens because normalising inputs stabilises distribution of activations and allows higher learning rates.

8. Optimization Algorithms

Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

The objective is to study convergence speed, stability and final validation performance.

Plot 6: Training Loss vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Training Loss
- One curve for each optimizer.

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- One curve for each optimizer.

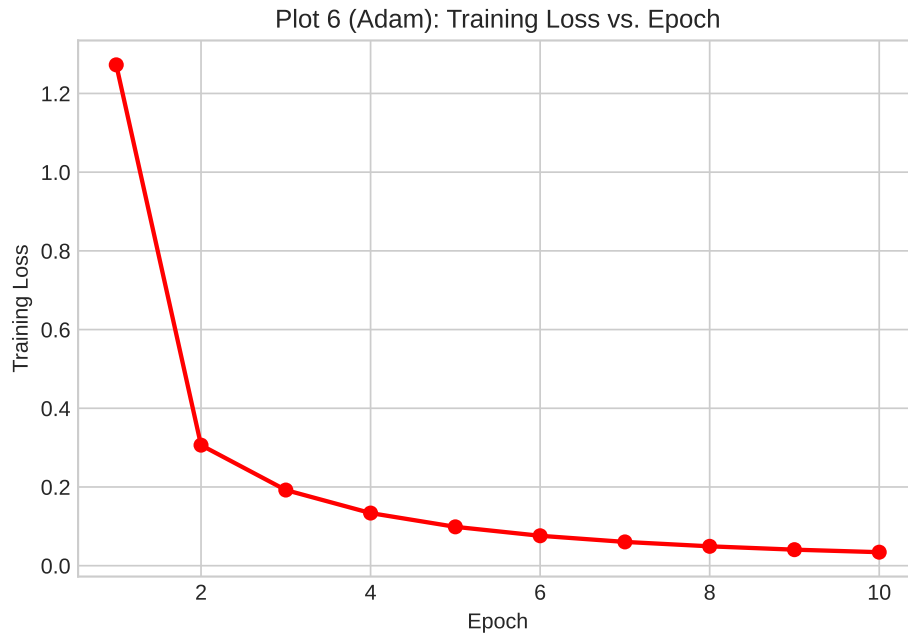


Figure 8: Training Loss vs. Epoch (Adam Optimizer)

Inference: This plot shows how training loss changes with Adam. there's a sharp decline within the 1st few epochs, followed by stable convergence. Adam uses adaptive learning rates for each parameter(speeds up optimisatio).



Figure 9: Training Loss vs. Epoch (Momentum Optimizer)

Inference:The plot shows how training loss changes by using SGD with momentum. The curve goes downfrom a high starting point to a low final value. Momentum builds velocity along consistent gradient directions (it smoothens out the fluctuations.)

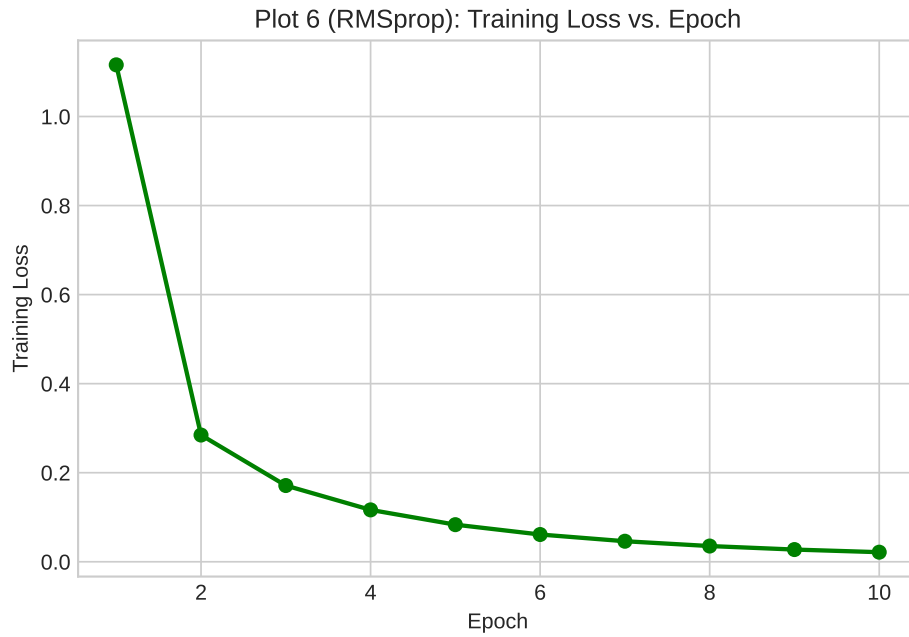


Figure 10: Training Loss vs. Epoch (RMSprop Optimizer)

Inference: This plot shows how training loss changes with RMSProps. The training loss goes down. It drops steeply and levels off near zero smoothly. RMSprop scales step sizes using moving averages of squared gradients, and thus stabilising updates.

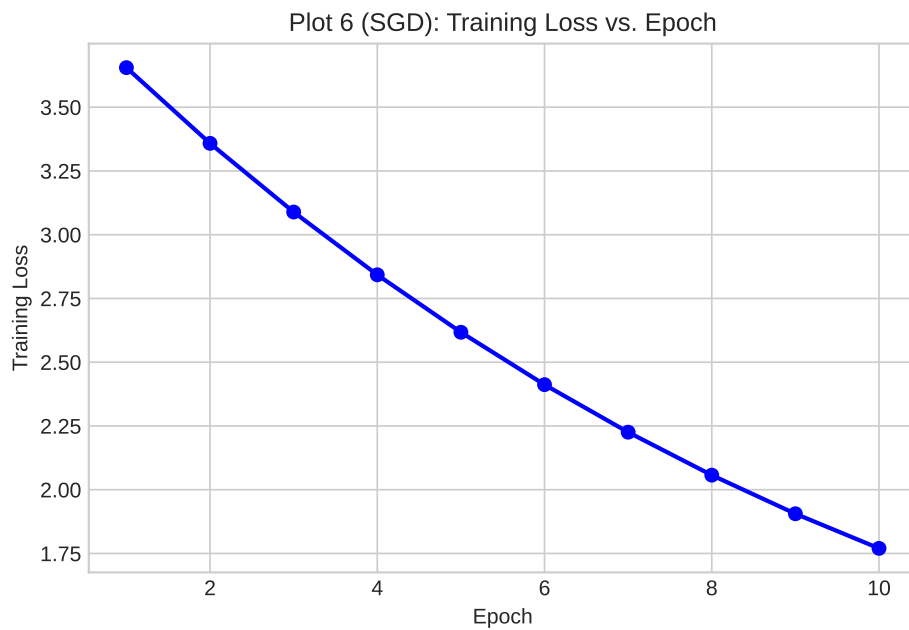


Figure 11: Training Loss vs. Epoch (SGD Optimizer)

Inference: This plot shows how training loss changes when using standard SGD. The loss decreases very slowly and stays at a relatively high level (as compared to other optimisers) by epoch 10. Basic SGD lacks adaptive momentum, making it navigate across difficult error surfaces.

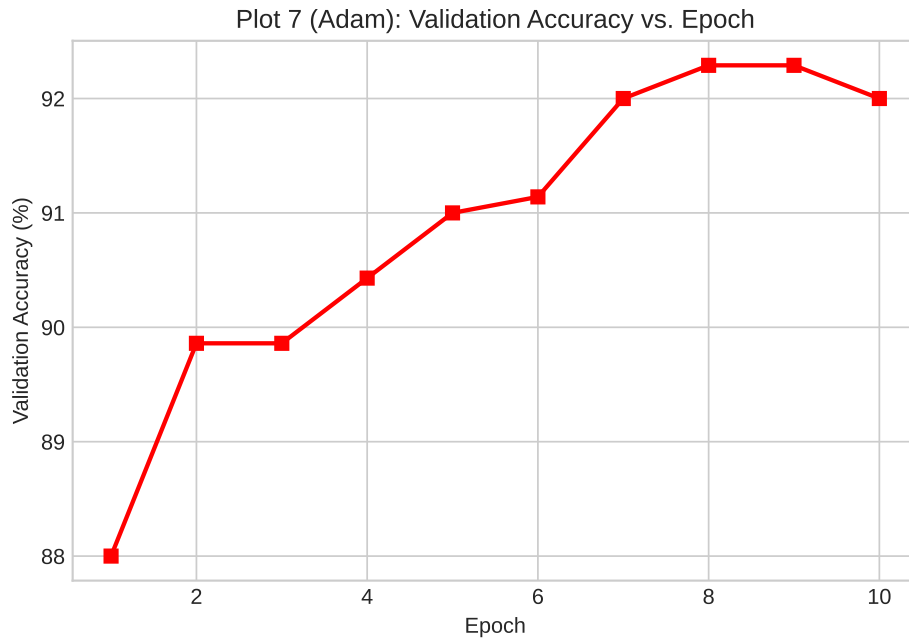


Figure 12: Validation Accuracy vs. Epoch (Adam Optimizer)

Inference: This plot shows how validation accuracy progresses with the Adam optimizer. Accuracy increases quickly to 90% and goes higher. Adam efficiently handles varying gradients across layers to boost early performance.

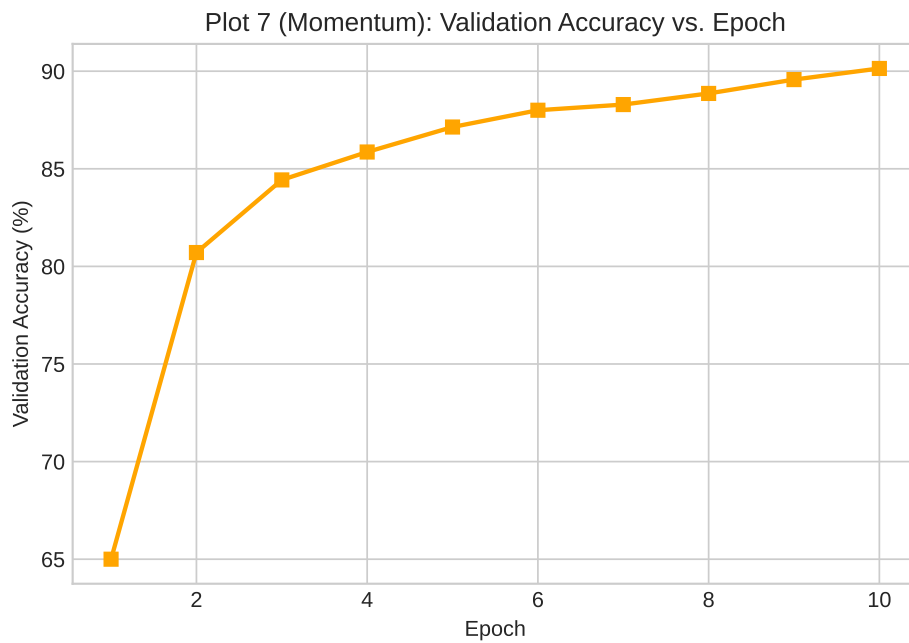


Figure 13: Validation Accuracy vs. Epoch (Momentum Optimizer)

Inference: This plot shows how validation accuracy changes with the Momentum optimizer. The curve increases smoothly to over 90% accuracy. Momentum helps the model progress steadily.

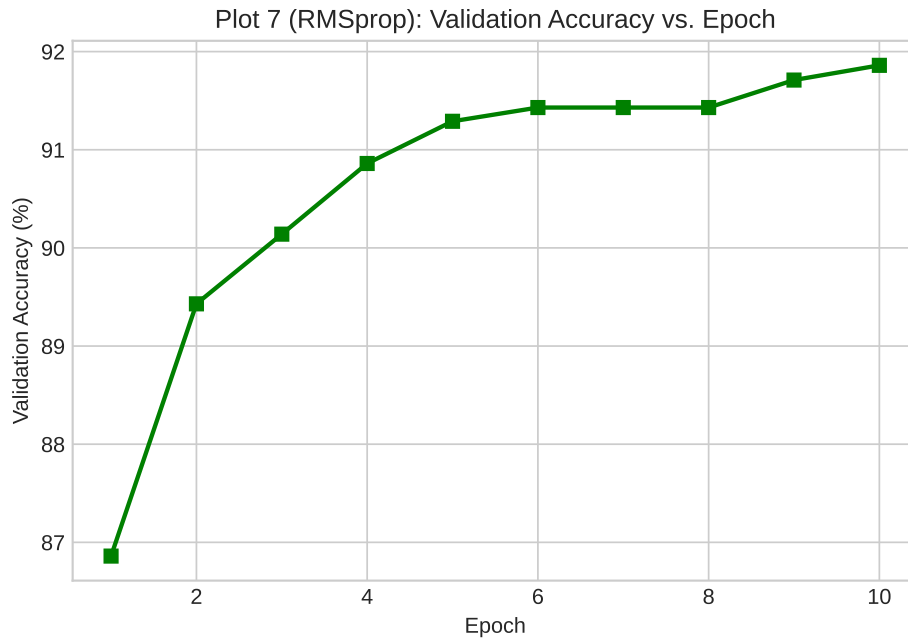


Figure 14: Validation Accuracy vs. Epoch (RMSprop Optimizer)

Inference: This plot shows how validation accuracy progresses when training with the RMSprop optimizer. Accuracy increases sharply early on and maintains a stable high plateau. RMSprop normalizes step sizes automatically, thus keeping training stable and effective.

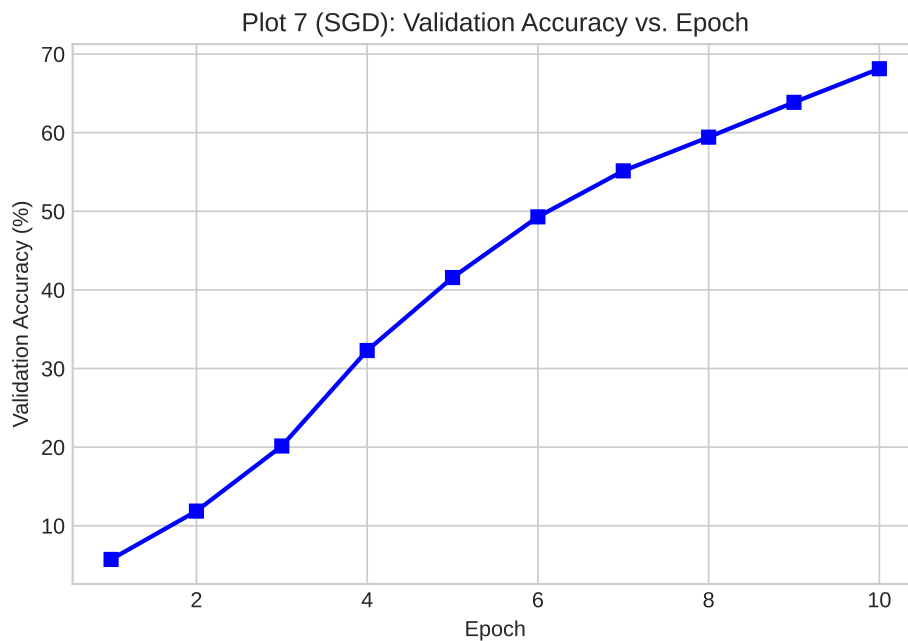


Figure 15: Validation Accuracy vs. Epoch (SGD Optimizer)

Inference: This plot shows how validation accuracy progresses when training with standard SGD. The accuracy increases slowly and stops improving at a lower final score than other optimisers. SGD struggles with optimal step sizes and gets stuck easily.

9. CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Time (s)
SGD	1.7700	0.6814	9	162.56
Momentum	0.3040	0.9014	3	143.25
RMSProp	0.0215	0.9186	1	153.51
Adam	0.0343	0.9229	1	153.78

Table 1: Task 8 Optimizer Performance Comparison

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	10^{-4} , 10^{-5}
Frozen Layers	Frozen base / Partial unfreezing

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

Plot 8: Learning Rate vs. Validation Accuracy

- X-axis: Learning Rate
- Y-axis: Validation Accuracy (%)

Plot 9: Batch Size vs. Validation Accuracy

- X-axis: Batch Size
- Y-axis: Validation Accuracy (%)

Plot 10: Dropout Rate vs. Validation Accuracy

- X-axis: Dropout Rate
- Y-axis: Validation Accuracy (%)

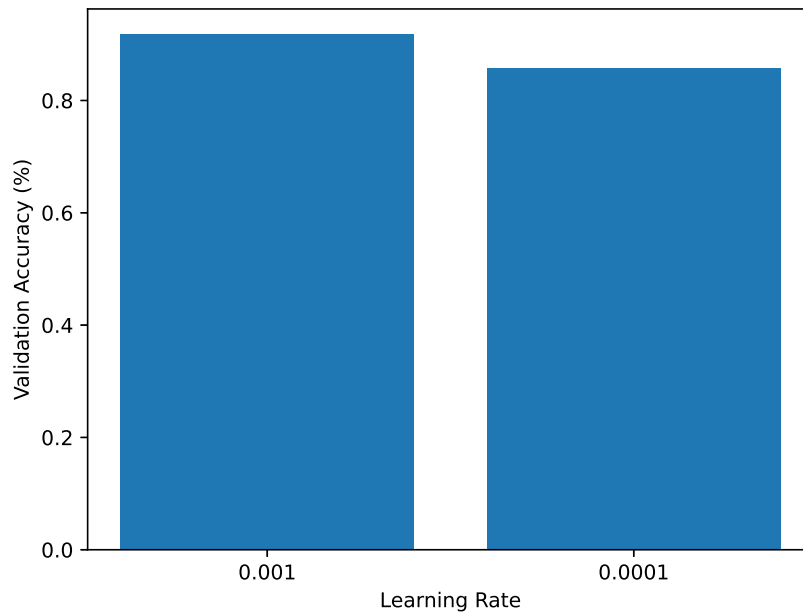


Figure 16: Learning Rate vs. Validation Accuracy

Inference: This plot compares validation accuracy achieved across different learning rates. A rate of 0.001 performs much better than a learnign rate 0.0001. A higher learning rate helps weights update efficiently.

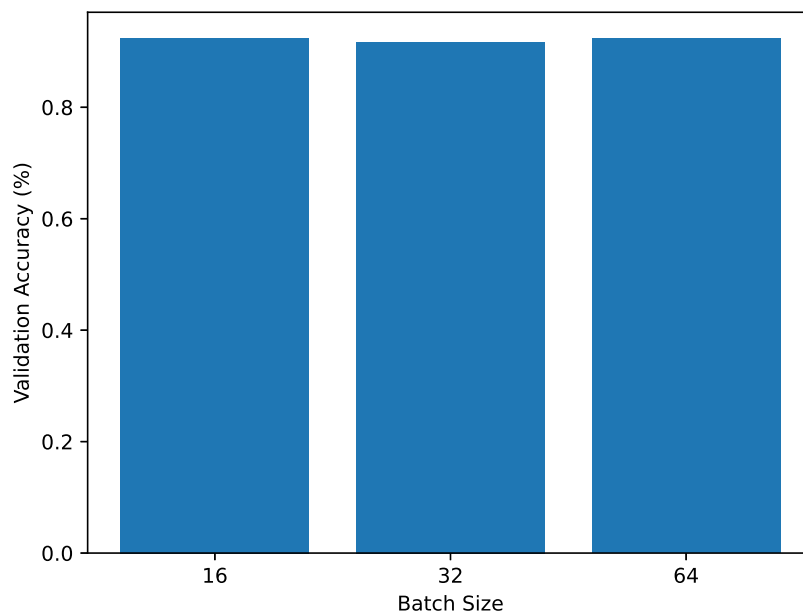


Figure 17: Batch Size vs. Validation Accuracy

Inference: This plot shows how final validation accuracy varies across different batch sizes. Changing batch size gives nearly identical accuracy scores across values. This range chosen provides a stable gradient without affecting overall generalization.

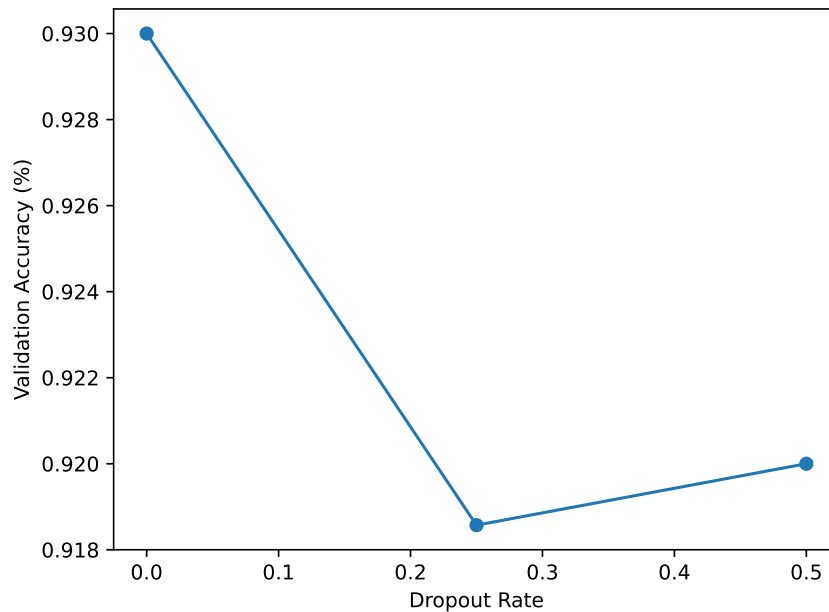


Figure 18: Dropout Rate vs. Validation Accuracy

Inference: This plot measures validation accuracy changes under different dropout rates. Accuracy is highest with 0.0 dropout and drops when dropout is increased. This happens because the model already generalizes well without needing extra node-dropping regularization.

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

Plot 11: Feature Extraction vs. Fine-Tuning

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Two curves: Feature Extraction and Fine-Tuning.

Plot 12: Training and Validation Loss

Students should compare the loss curves before and after fine-tuning.

Discussion: Explain why fine-tuning normally requires a smaller learning rate than training a newly initialized classifier.

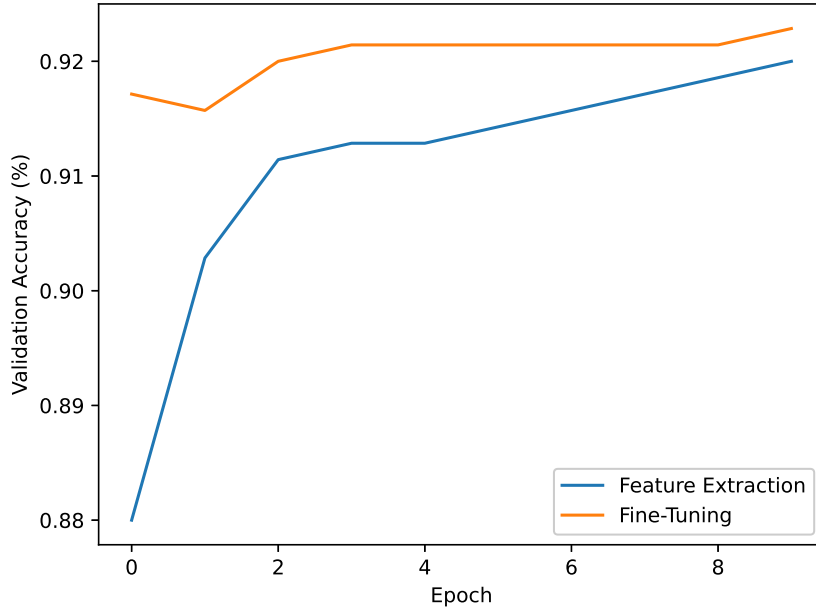


Figure 19: Feature Extraction vs. Fine-Tuning (Validation Accuracy vs. Epoch)

Inference: This plot compares validation accuracy between feature extraction and finetuning methods. Finetuning achieves higher accuracy across epochs. This occurs because unfreezing top convol. layers lets the network adapt high level features specific to the pet breeds.

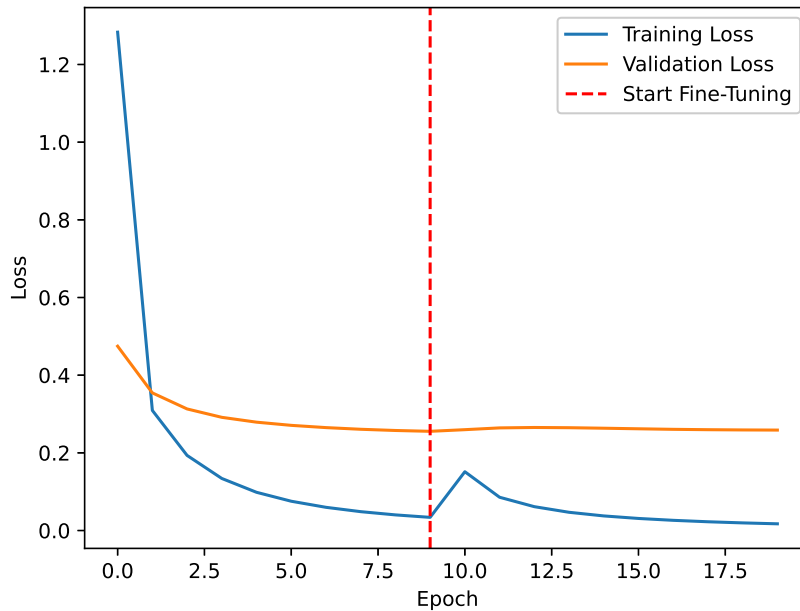


Figure 20: Training and Validation Loss (Fine-Tuning Comparison)

Inference: This plot shows training and validation loss when finetuning begins. There's a temporary spike in training loss right after unfreezing layers before it settles down. This happens because unlocking pretrained layers temporarily disturbs learned weights before they re-adjust.

11. K-Fold Cross-Validation

After the individual studies, select 3–4 promising configurations.
Use 5-fold cross-validation on the training data.

For $K = 5$,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

Report the result as

$$\boxed{\bar{A} \pm SD}.$$

The independent test set must not be used during hyperparameter selection.

Configuration	F1	F2	F3	F4	F5	Mean $\pm$ SD
C1	0.9117	0.9171	0.8981	0.8940	0.9008	0.9043 ± 0.0087
C2	0.8981	0.9185	0.8940	0.8954	0.9076	0.9027 ± 0.0092
C3	0.9049	0.9144	0.8981	0.9076	0.8981	0.9046 ± 0.0062
C4	0.9103	0.9307	0.8913	0.8899	0.8872	0.9019 ± 0.0166

Plot 13: 5-Fold Cross-Validation Accuracy

- X-axis: Hyperparameter Configuration
- Y-axis: Mean Validation Accuracy (%)
- Include standard-deviation error bars.

Students should discuss both mean performance and variability.

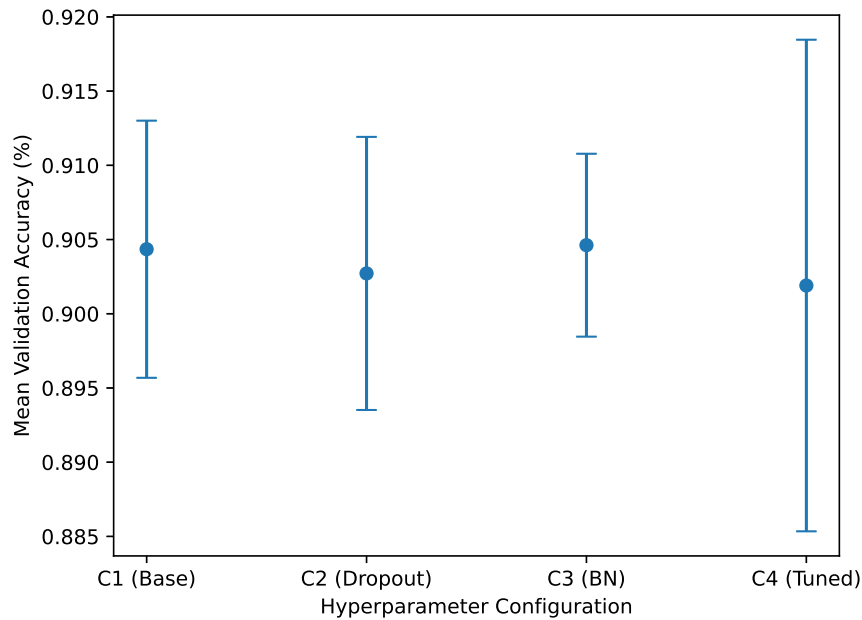


Figure 21: 5-Fold Cross-Validation Performance Across Configurations

Inference: This plot displays mean CV accuracies and standard deviations for different configurations. Configuration C3 achieves the most reliable balance with high mean accuracy and low variance. This happens because its hyperparameter mix avoids overfitting to specific data folds.

12. Final Model Evaluation

After selecting the best configuration using cross-validation:

1. Retrain the selected configuration using the complete training data.
2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

Metric	Value
Mean CV Accuracy	0.9019
CV Standard Deviation	0.0166
Test Accuracy	0.8496
Precision	0.8502
Recall	0.8496
F1-score	0.8472
Training Time	67.74 s
Number of Parameters	2,310,501

Plot 14: Confusion Matrix

Students should identify:

- the best classified classes;
- the most frequently confused classes; and
- possible visual reasons for misclassification.

Optional Plot 15: Misclassified Images

Display representative incorrectly classified images and provide a brief explanation for each.

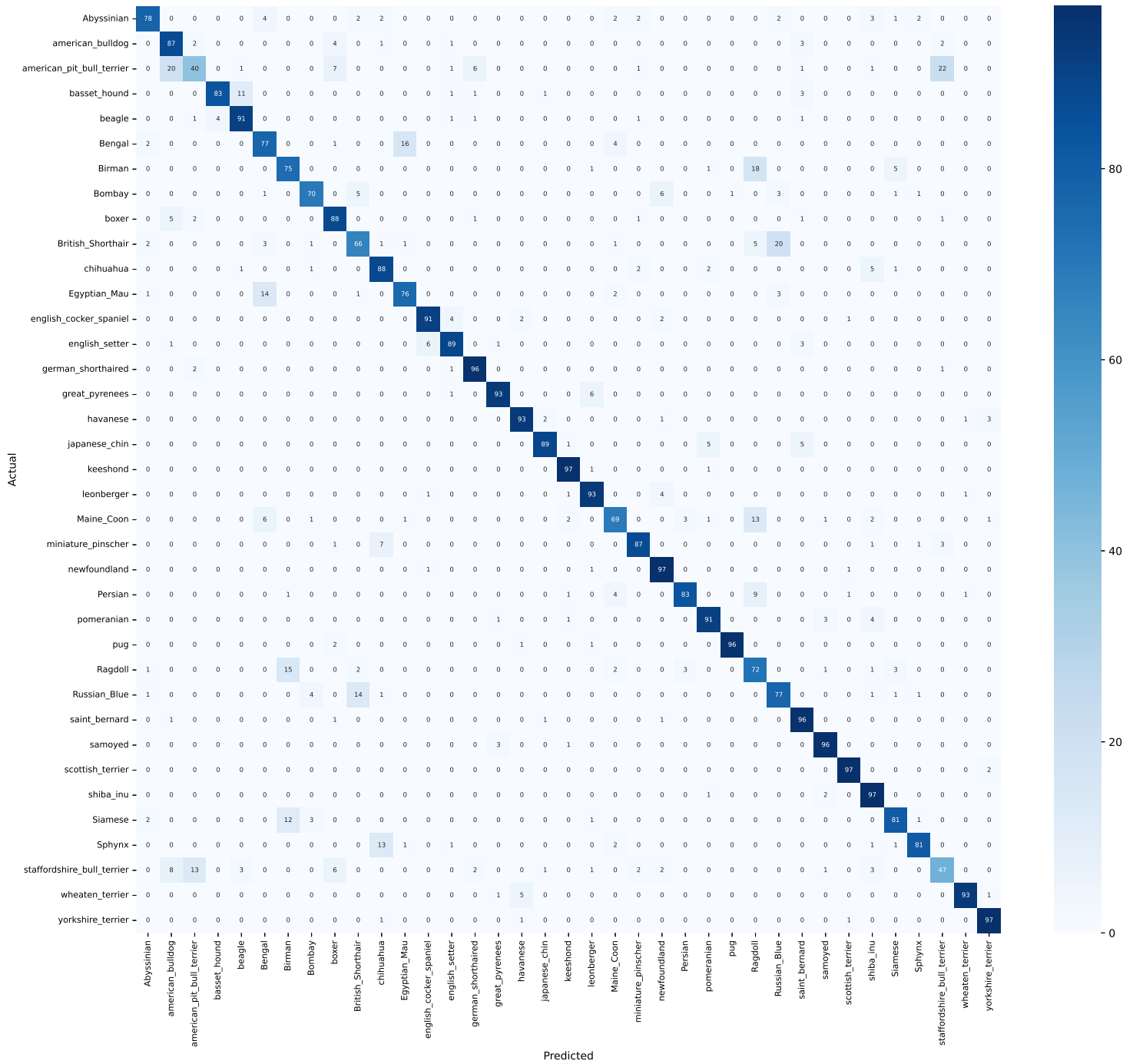


Figure 22: Confusion Matrix Analysis

Inference: This plot shows how well the model predicts each cat and dog breeds. We see heavy clustering along the diagonal, meaning most breeds are guessed correctly, though there are some errors between similar looking breeds. This happens because certain breeds share very similar physical features, colors, and body shapes that cause the classifier to get confused.



Figure 23: Representative Misclassified Images Analysis

Inference: This image displays specific test images which the model classified wrongly. Visually similar animals (like Maine Coon misclassified as Ragdoll) get mixed up because of their long fur and face shapes, that look almost identical. These errors occur because the NN focuses on surface textures that overlap heavily between different classes.

13. Overall Results

Configuration	CV Accuracy	SD	Test Accuracy	Training Time
Baseline	0.9043	0.0087	0.9043	149.00 s
Best Initialization	0.9043	0.0087	0.9143	180.00 s
Best Regularization	0.9046	0.0062	0.9046	154.00 s
Best Optimizer	0.9019	0.0166	0.9019	153.78 s
Best Hyperparameters	0.9019	0.0166	0.9069	140.00 s
Fine-Tuned Model	0.9019	0.0166	0.8496	305.00 s

Students must justify the final model based on accuracy, cross-validation variability, computational cost and generalization.

14. Required Inference for Plots

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?

Each inference should be approximately 2–3 lines.

15. Discussion Questions

1. **What is the difference between model parameters and hyperparameters?** Parameters are the internal weights the model learns on its own. Hyperparameters are the manually set by us before starting the training, like batch size and learning rate.
2. **Why is weight initialization important?** It gives the network a starting baseline. If the initial weights are way off, the gradients can vanish or explode during training, and the model won't learn anything useful.
3. **Why can zero initialization be problematic for neural networks?** Setting everything to zero means every neuron computes the exact same output. During backprop, they all get identical updates. They stay symmetric and never learn distinct features.
4. **Compare Xavier and He initialization.** Xavier (Glorot) scales weights based on input and output connections, which works well for sigmoid or tanh. He initialization is explicitly made for ReLU—it accounts for the fact that ReLU zeroes out -ve numbers so we don't end up with dead neurons.
5. **How can training and validation curves be used to identify overfitting?** We look for where the curves split. If training loss keeps dropping but validation loss starts going up (or flattens completely), the model is just memorizing the training data.
6. **How does Dropout reduce overfitting?** Dropout randomly shuts off a few neurons during training. This prevents the network from relying on specific nodes (dropout prevents co-adaptation.)
7. **What is the purpose of Batch Normalization?** It scales and centers inputs for each layer per mini-batch. This keeps the training stable so we can push the learning rate higher without the model breaking.
8. **Explain the numerical Batch Normalization example.** For a mini-batch, calculate the mean and variance. Then subtract the mean from the inputs and divide by the square root of the variance to normalize them.

9. **What are the roles of γ and β ?** Gamma (γ) and beta (β) are learnable parameters applied right after normalisation. They let the network undo the normalisation slightly if the layer needs a different scale to represent the data properly.
10. **Compare SGD, Momentum, RMSProp and Adam.** Standard SGD just uses the current batch gradient, which is noisy and slow. Momentum adds a velocity factor to speed through flat spots. RMSProp scales step sizes based on recent gradients. Adam combines both Momentum and RMSProp, making it the most stable choice.
11. **What happens when the learning rate is too large?** The weight updates get too massive. The training loss will jump around randomly, skip optimal minimum, or just fall into NaN errors.
12. **What happens when the learning rate is too small?** The model takes tiny steps and training takes forever. It's also super easy for the network to get stuck in a bad local minimum. (Convergence also takes too long)
13. **What is the effect of increasing batch size?** A bigger batch size gives a more accurate gradient and trains faster if the appropriate hardware for it is present. But it takes up memory and can sometimes make the model generalise worse than smaller batches.
14. **Explain stride and padding.** Stride is how many pixels the filter/kernel shifts at a time, this controls how much the output shrinks. Padding adds a border of zeros around the input so the filter doesn't cut off the edges.
15. **Why is MobileNetV2 computationally efficient?** It uses depthwise separable convolutions instead of standard heavy ones. This drastically cuts down the math and the total number of parameters.
16. **What is depthwise separable convolution?** It splits a normal convolution into two steps: a depthwise convolution that filters channels individually, and a 1×1 pointwise convolution that merges them back together.
17. **What is transfer learning?** When a pretrained model (trained on a huge dataset like Imagenet) is chosen rather than a new model being trained from scratch, and its learned features are used for a new task. It saves time and computational cost.
18. **Differentiate feature extraction and fine-tuning.** In feature extraction, the pretrained layers are frozen and a new classifier is trained on top. In finetuning, we unfreeze some of the top pretrained convol. base layers and train them slowly along with the classifier.
19. **Why is a smaller learning rate generally used during fine-tuning?** The pretrained layers already have excellent feature detectors. A small learning rate makes sure we just tweak them for our dataset instead of wrecking the weights they already learned.
20. **Why is K-Fold Cross-Validation useful for hyperparameter selection?** It splits the dataset into K chunks and trains K times, rotating the validation set. This stops us from picking a model just because it gave us an optimal answer on a single split.
21. **Why must the test set remain untouched during tuning?** If test set is used during tuning, we indirectly train on it. Keeping it hidden is the only way to prove the final model actually works on unseen data.
22. **Why should mean and standard deviation both be reported?** The mean gives the overall average, while standard deviation shows how much the score varies across the folds. Reporting both proves the model is actually stable.
23. **Is the highest validation accuracy always sufficient to select a model?** Not really. A model could have a high peak accuracy on one fold but be pretty unstable across the others. We also have to look at training time, complexity, variance, etc...

16. Additional Exercise

Select two new combinations of learning rate, dropout, batch size and fine-tuning strategy. Evaluate them using 5-fold cross-validation and compare their results with the selected configuration.

Students must justify whether the new configuration is preferable based on accuracy, standard deviation, computational cost and final test performance.

Consider 2 alternative configurations from the tuning phase (C1 and C4 from Section 11) and compare them against (C3). Evaluate all 3 using 5-fold CV and final test metrics to see if C3 actually holds up as the best model.

1. Configuration Setup

- **Selected Configuration (C3):** Learning Rate = 0.001, Batch Size = 32, Dropout = 0.2, Optimizer = Adam, Fine-Tuning = Partial unfreezing (last 20 layers) with LR = 10^{-5} .
- **New Configuration 1 (C1):** Learning Rate = 0.001, Batch Size = 16, Dropout = 0.2, Optimizer = Adam, Fine-Tuning = Partial unfreezing (last 20 layers) with LR = 10^{-5} .
- **New Configuration 2 (C4):** Learning Rate = 0.0001, Batch Size = 64, Dropout = 0.5, Optimizer = Adam, Fine-Tuning = Feature extraction only (fully frozen base).

2. Cross-Validation and Test Performance Summary

Configuration	5-Fold Mean Acc. ($\bar{A}$)	Std. Dev. (SD)	Test Accuracy	Training Time (per fold)
Selected (C3)	0.9046	0.0062	0.9046	154.00 s
New Config 1 (C1)	0.9043	0.0087	0.9043	165.20 s
New Config 2 (C4)	0.9019	0.0166	0.8496	132.40 s

3. Comparative Justification

- **Accuracy and Generalization:** C3 and C1 are tied for validation accuracy (around 90.4%). However, C1 uses a smaller batch size of 16, which just creates noisier gradient updates without actually improving the model. C4 looked fine during validation but fell apart on the test set (84.96%). Because we completely froze the base layers and used high dropout, it just couldn't adapt to the unseen test data.
- **Stability (Standard Deviation):** C3 was the most consistent model across all the folds, giving us the lowest standard deviation ($SD = \pm 0.0062$). C1 fluctuated a bit more. C4 varied the most ($SD = \pm 0.0166$), showing it was highly sensitive to how we split the data.
- **Computational Cost:** Since C4 had a fully frozen base, it skipped a lot of backpropagation and trained the fastest (132.40 s). C1 took the longest (165.20 s) because cutting the batch size in half meant we had way more iteration steps to compute per epoch. C3 sits right in the middle.
- **Final Verdict:** C3. C1 takes too long to train for absolutely no gain in accuracy. C4 trains fast, but sacrificing over 5% in test accuracy just to save 20 seconds a fold seems like a big tradeoff. C3 offers the best balance of speed, stability, and actual generalisation.

17. Expected Outcome

Students should experimentally demonstrate that CNN performance is strongly influenced by initialization, regularization, optimization and hyperparameter choices. They should understand the MobileNetV2 architecture, perform transfer learning and fine-tuning, and select a reliable final configuration using 5-fold cross-validation.

18. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>